

Serving — Kimi K2 Thinking, Inference Scheduler, and API Access

Rev 0.1 · 2026-07-19

Generated from Markdown — edit the `.md` source and regenerate (`python3 docs/md2pdf.py`)

Contents

1 Model choice — Kimi K2 Thinking	3
2 Parallelism layout — TP=8 in-node, replicas across nodes	4
2.1 Serving engine	4
3 Request flow (SPEC §8, realized)	5
3.1 Why a scheduling layer at all	5
4 API surface	6
5 Observability and SLOs	7
6 Failure behavior	8
7 Phase-next	9
8 Verification gates	10
9 Revision History	11

Status: Draft

Scope: Production model (Kimi K2 Thinking) deployment, request scheduling, external API surface, auth/quotas, capture pipeline wiring

Implements: [SPEC.md](#) §4 (parallelism), §6.6–6.8 (model storage + capture), §7 (K8s stack), §8 (request flow)
· rides [docs/overlay.md](#) §4 for ingress

Manifests: `bootstrap/platform/apps/models/` + `bootstrap/platform/apps/api/` (Flux: `platform-models`, `platform-api`)

1 Model choice — Kimi K2 Thinking

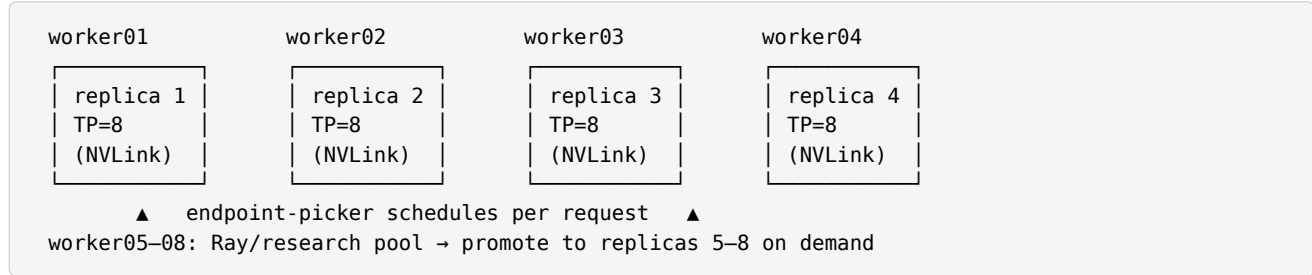
Latest Kimi release (Moonshot AI, Nov 2025) — a reasoning MoE that matches the cluster’s design target (“multi-trillion-class MoE, interactive”):

Property	Value	Fit
Architecture	MoE, 1T total / 32B active params, 384 experts (8 routed + 1 shared per token), MLA attention	exactly the SPEC §4 workload class
Weights	native INT4 QAT, ~594 GB (compressed-tensors)	fits a <i>single</i> XE9680’s 1.4 TB HBM with room for KV
Context	256K tokens	exceeds the §8.1 32k planning floor
Interface	OpenAI-compatible (chat, tools, reasoning content)	vLLM serves it directly
License	Modified MIT	research use OK

HF id: moonshotai/Kimi-K2-Thinking. Promotion follows SPEC §6.8: immutable /models/kimi-k2-thinking/v2026-07-19/ + CHECKSUMS, atomic current symlink (staged by the kimi-k2-thinking-download Job).

2 Parallelism layout — TP=8 in-node, replicas across nodes

SPEC §4.3 allows TP=8, EP=8 across nodes; K2 Thinking’s INT4 footprint makes the simpler layout strictly better day-1:



Decision	Rationale
TP=8 inside one node	594 GB + KV fits in 1.4 TB HBM; TP traffic stays on NVLink — zero rail-fabric dependency per token
Replica = failure domain	node loss removes one replica; scheduler drains to the rest (SPEC §12 alignment)
No cross-node EP day-1	wide-EP buys throughput, not latency, and couples every token to the fabric; revisit when concurrency outgrows 8 replicas
4 replicas day-1 (32 GPUs)	~50 concurrent $\approx$ 12-16 streams/replica at 256K-capable KV headroom; workers 05-08 stay a Ray/research pool until promoted

Capacity check against SPEC §8.2: 4 replicas $\times$ ~300-500 tok/s aggregate each (32B-active INT4 on 8 $\times$ B200) $\geq$ 1,000 tok/s cluster target with interactive TPOT.

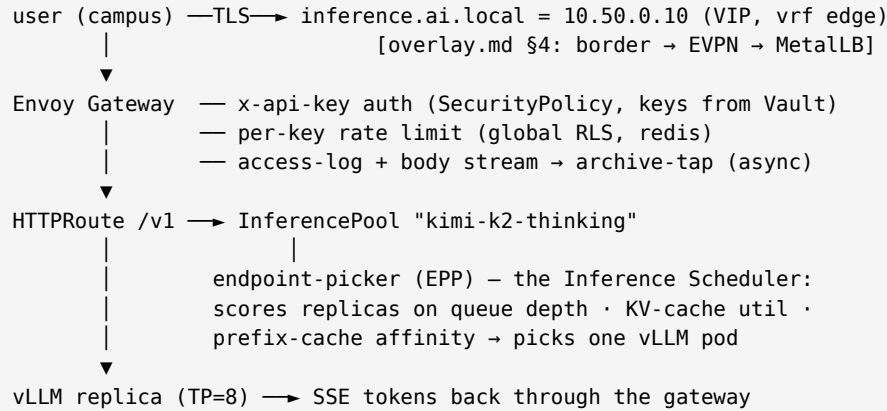
2.1 Serving engine

vLLM (registry.ai.local/library/vllm-openai:v0.11.0, KServe RawDeployment):

```
--model=/models/kimi-k2-thinking/current --served-model-name=kimi-k2-thinking
--tensor-parallel-size=8 --max-model-len=262144 --max-num-seqs=32
--gpu-memory-utilization=0.92 --trust-remote-code
--enable-auto-tool-choice --tool-call-parser=kimi_k2 --reasoning-parser=kimi_k2
```

Weights load from the hot FS (§6.4: $\geq$ 10-20 GB/s per node $\rightarrow$ sub-minute read; engine init dominates). Rollout = new version dir + symlink flip + KServe rolling restart, one replica at a time.

3 Request flow (SPEC §8, realized)



SPEC §8 box	Implementation
API Gateway	Envoy Gateway (apps/api/envoy-gateway.yaml), VIP via MetalLB in vrf edge
AuthN/AuthZ	API-key SecurityPolicy + Vault-sourced key set, 5-min revocation propagation
Inference Scheduler	Gateway API Inference Extension endpoint-picker (apps/api/inference-scheduler.yaml)
Model Server	KServe InferenceService kimi-k2-thinking (apps/models/kimi-k2.yaml)
Capture	archive-tap → Redpanda → hourly parquet compactor → object lake (apps/api/archive.yaml)

3.1 Why a scheduling layer at all

Round-robin over LLM replicas is actively bad: a replica with a deep queue or full KV cache stalls TTFT for everyone routed there while its neighbor idles. The EPP consults live vLLM metrics per request and also gives us, for free later: criticality-based shedding (`InferenceModel.criticality`), LoRA-aware routing, and multi-model pools when a second model ships.

4 API surface

OpenAI-compatible, served by vLLM end-to-end — no translation layer:

Endpoint	Notes
POST /v1/chat/completions	streaming + tools + reasoning_content (K2 Thinking)
POST /v1/completions	legacy
GET /v1/models	returns kimi-k2-thinking

Client bootstrap:

```
client = OpenAI(base_url="https://inference.ai.local/v1",
                 api_key="unused",                      # auth is the header below
                 default_headers={"x-api-key": "<key>"})
r = client.chat.completions.create(model="kimi-k2-thinking",
                                   messages=[...], stream=True)
```

Key lifecycle: mint/revoke in Vault (secret/inference/api-keys), ESO syncs to the gateway within 5 min. Default quota 120 req/min/key (auth.yaml); token-based metering is the phase-next upgrade (§7).

5 Observability and SLOs

Signal	Source	SLO direction (SPEC §8.1/§11.3)
TTFT / TPOT / queue depth / KV util	vLLM / metrics via PodMonitor	TTFT p50 interactive; TPOT ≥ 20 tok/s/user
Per-key re- quest rate, 401/429s	Envoy metrics + access logs	quota health, abuse detection
Scheduler de- cisions	EPP metrics	balanced replica load, no hot replica
Archive lag lag on <code>inference.records</code>	tap/consumer	$p99 \leq 60$ s (SPEC §6.4)
GPU health	DCGM (gpu- operator)	SPEC §11.1

Formal SLO doc remains `docs/slo.md` (roadmap).

6 Failure behavior

Event	Behavior
Replica/node loss	EPP stops selecting it; in-flight streams on it drop (client retry); capacity −25% at 4 replicas
All gateway pods lost	VIP unreachable; MetalLB withdraws on node loss; 2 proxy replicas across nodes
Redpanda down	archive-tap spools to NVMe ring (<code>inference-archive-scratch</code>), replays; tokens unaffected
Vault down	existing keys keep working from synced Secret; mint/revoke paused
Hot FS down	live serving unaffected (weights in HBM); new replica starts fail (SPEC §6.11)
Bad model promote	flip <code>current</code> symlink back, rolling restart (immutable versions)

7 Phase-next

Item	Trigger
Envoy AI Gateway layer	need token-based quotas/billing, Bearer-key auth, multi-provider fallback
Wide-EP (DP+EP across nodes, DeepEP-class kernels)	concurrency beyond 8 TP=8 replicas, or batch-throughput tier
Second model in the pool	research ask; add InferenceService + InferenceModel, same pool pattern
KV/prefix-cache spill to local NVMe	long-context multi-turn dominates (SPEC §6.1)
PII/redaction hooks in archive-tap	policy requirement before durable write (SPEC §6.7)

8 Verification gates

Gate	Method	Pass
Weights staged	download Job complete; CHECKSUMS verifies; current → v2026-07-19	✓
Replica up	kubectl get isvc kimi-k2-thinking Ready; /v1/models on pod	✓
Scheduler path	100 parallel prompts → EPP metrics show spread across all replicas	no hot replica
External path	campus curl + streamed chat via VIP (overlay.md §6 gate)	200, tokens stream
Auth	no key → 401; revoked key → 401 within 5 min; over-quota → 429	✓
Capture	send tagged request; find it in <code>inference.records</code> , then in parquet prefix within the hour	request_id match
SLO smoke	50 concurrent synthetic users, 32K prompts	TTFT/TPOT within §5 targets

9 Revision History

Version	Date	Notes
0.1	2026-07-19	Initial: Kimi K2 Thinking (TP=8 × 4 replicas), EPP inference scheduler, Envoy gateway + API keys + quotas, archive tap wiring